Wilo-Pumpe Teil 3: PIO misst alles

Hardware-Pulsmessung mit dem RP2040-PIO — und was passiert, wenn 125 MHz zu schnell sind

make Magazin · Projekt-Doku

April 2026 `github.com/gerontec/wilo_pwm`

Teil 1 nutzte C-Interrupts, Teil 2 Core 1-Polling. Teil 3 delegiert die gesamte Messung an den Hardware-PIO — kein CPU-Overhead, kein Debounce, keine Firmware-Aenderung noetig. Aber auch PIO hat Tuecken: EMI vom Pumpenmotor und Abtastrate muessen zusammenpassen. Ergebnis: ein dynamisch robuster Median-Filter als rein Python-basiertes Modul.

---

1 Warum PIO besser als Core-1-Polling ist

In Teil 2 lief eine enge Polling-Schleife auf Core 1 mit 26 Millionen Iterationen pro Sekunde. Das funktioniert, hat aber zwei Schwaechen:

- **Software-Debounce noetig.** Jeder Glitch auf der Leitung muss manuell gefiltert werden — mit adaptivem Schwellwert, der bei Frequenzaenderungen neu berechnet wird.
- **Firmware-Build erforderlich.** Das Core-1-Modul ist als `USER_C_MODULES` in die Firmware gelinkt — jede Aenderung erfordert einen neuen Build und BOOTSEL-Flash.

Der RP2040-PIO loest beide Probleme auf einmal. PIO (Programmable I/O) ist ein eigenstaendiger Prozessor mit eigenem Instruktionssatz und eigenem Takt, voellig unabhaengig von ARM-Core 0 und Core 1. Ein PIO-Programm laeuft deterministisch Zyklus fuer Zyklus, wird nicht von GC-Pausen unterbrochen und braucht keine IRQ-Vektoren. Und das Beste: In MicroPython laesst sich PIO als reines Python-Modul deployen — kein Compiler, kein Flash.

2 Das PIO-Assembler-Programm

Das Messprinzip ist direkt:

1. Auf steigende Flanke warten (`wait`)
2. Abwaertszaehler X von `0xFFFFFFFF` herunterzaehlen solange der Pin HIGH ist
3. Zaehlerstand in RX-FIFO schieben (`push`)
4. Dasselbe fuer die LOW-Phase wiederholen

Listing 1: pwmfeedback_pio.py — PIO-Programm fuer HIGH/LOW-Messung

```
@rp2.asm_pio()
def _pwm_measure():
    pull()            # Y-Initialwert (0xFFFFFFFF) aus TX-FIFO laden
```

```
    mov(y, osr)

    wrap_target()

    wait(0, pin, 0)  # Sync: warten bis Pin LOW
    wait(1, pin, 0)  # warten auf steigende Flanke (Pin wird HIGH)

    # HIGH-Zeit messen: X abwaerts zaehlen solange Pin HIGH
    mov(x, y)                    # X = 0xFFFFFFFF
    label("high_loop")
    jmp(pin, "high_count")       # Pin HIGH? -> weiterzaehlen
    jmp("high_done")             # Pin LOW?  -> fertig
    label("high_count")
    jmp(x_dec, "high_loop")      # X--, und zurueck
    label("high_done")
    mov(isr, x)
    push()                       # blockierend: stalliert wenn FIFO voll

    # LOW-Zeit messen: X abwaerts zaehlen solange Pin LOW
    mov(x, y)
    label("low_loop")
    jmp(pin, "low_done")         # Pin HIGH? -> LOW-Phase vorbei
    jmp(x_dec, "low_loop")       # X--, weiterzaehlen
    label("low_done")
    mov(isr, x)
    push()                       # blockierend: HIGH+LOW immer als Paar

    wrap()
```

Das Programm besteht aus 15 Instruktionen — weit unter dem Limit von 32. Die Zaehlschleife benoetigt **2 Zyklen pro Schritt**. Bei einem PIO-Takt von 1 MHz ergibt das eine Zeitaufloesung von 2 µs.

3 State Machine und Python-Auswertung

Die State Machine wird mit einem einzigen `put()` initialisiert, der den Startwert 0xFFFFFFFF in den TX-FIFO legt:

Listing 2: Initialisierung der State Machine

```
pin = Pin(5, Pin.IN, Pin.PULL_UP)
sm = rp2.StateMachine(
    0, _pwm_measure,
    freq=1_000_000,     # 1 MHz -> 1 us/Zyklus, 2 us/Zaehlschritt
    in_base=pin,
    jmp_pin=pin,
)
sm.put(0xFFFFFFFF)      # Y-Initialwert
sm.active(1)
```

Die Python-Seite liest HIGH- und LOW-Wert paarweise aus dem FIFO und rechnet Zeitdauer, Frequenz und Duty-Cycle:

Listing 3: Auswertung: Frequenz und Duty-Cycle aus Zaehlerstand

```
x_high = sm.get()
x_low  = sm.get()
# (0xFFFFFFFF - x) * 2 us/Count = Pulsdauer in Mikrosekunden
```

```
high_us = (0xFFFFFFFF - x_high) * 2
low_us  = (0xFFFFFFFF - x_low)  * 2
T       = high_us + low_us
freq    = 1_000_000.0 / T        # Hz
duty    = high_us / T * 100.0    # Prozent
```

`push()` (blockierend) sorgt dafuer, dass HIGH und LOW stets als vollstaendiges Paar im FIFO landen — die State Machine stalliert lieber, als einen einzelnen Wert zu verwerfen und die Synchronisation zu brechen. Warum das entscheidend ist, erklaert Abschnitt 11.

4 Warum 125 MHz nicht funktioniert: das EMI-Problem

Der erste Versuch lief die State Machine mit 125 MHz (Systemtakt) — hoehere Aufloesung, dachte man. Das Ergebnis: die gemessene Frequenz betrug konstant 181 Hz statt 75 Hz, der Duty-Cycle war 80–88 % statt 30–65 %.

Die Ursache liegt in der Abtastrate. Bei 125 MHz prueft die State Machine den Pin alle **8 ns**. Pumpen-Elektronik erzeugt durch Motorkommunikation und PWM-Schalten breitbandige elektromagnetische Stoerungen (EMI) mit Impulsdauern von typisch 50–500 ns. Bei 8 ns Abtastrate sind solche Gliches sichtbar:

Listing 4: LOW-Schleife bei 125 MHz: EMI-Glitch beendet Messung

```
label("low_loop")
jmp(pin, "low_done")    # Pin HIGH? -> LOW-Phase vorbei
jmp(x_dec, "low_loop") # X--, weiterzaehlen
```

Ein 100 ns-Spike waehrend der LOW-Phase trifft mit hoher Wahrscheinlichkeit eine der `jmp(pin)`-Instruktionen und beendet die LOW-Messung vorzeitig. Statt 9 333 µs LOW-Zeit wird 988 µs gemessen — Faktor 9 zu wenig.

Loesungsversuche: Ein Inline-Debounce im PIO-Programm (mehrfaches Bestaetigen der Flanke) wurde implementiert und getestet — 7 Checks a 8 ns = 56 ns Debounce fuer die HIGH-Schleife, 6 Checks a 8 ns = 48 ns fuer die LOW-Schleife. Das PIO-Programm wuchs auf 31 Instruktionen (Limit: 32). Ergebnis: weiterhin instabil. Pumpenmotor-EMI ist schlicht breitbandiger als 56 ns.

Die richtige Loesung: 1 MHz-Takt. Bei 2 µs Abtastperiode sind alle Gliches unter 2 µs physikalisch unsichtbar — ein kostenloser Tiefpassfilter durch die Abtastrate selbst. Fuer 75 Hz ist 1 MHz 3000-fach ueberdimensioniert — die Aufloesung von 2 µs reicht absolut aus.

5 Frequenzlimit: Wann stoesst PIO an seine Grenzen?

Die Zaehlschleife benoetigt 2 PIO-Zyklen pro Schritt. Bei einem PIO-Takt von 1 MHz gilt:

$$t_{\min} = 2 \times \frac{1}{f_{PIO}} = 2\,\mu s$$

Fuer ein Signal mit 50 % Duty-Cycle muss sowohl HIGH als auch LOW mindestens 2 µs betragen — also Periode $\geq$ 4 µs, Frequenz $\leq$ **250 kHz**.

PIO-Takt	Aufloesung	Max. Freq. (50% Duty)
1 MHz	2 µs	250 kHz
10 MHz	200 ns	2,5 MHz
125 MHz	16 ns	~31 MHz

Hinweis: Hoehere Taktfrequenzen sind technisch moeglich, aber bei industriellen Umgebungen (Motoren, Schaltnetzteile) erfordert jeder Takt ueber ca. 500 kHz einen Hardware-Tiefpassfilter (z. B. 100 + 10 nF) vor dem GPIO-Pin, um Motorspikes zuverlaessig zu unterdruecken.

6 Robuste Messung: Ringpuffer und Median-Filter

Die einfache Einzel-Messung hat einen weiteren Schwachpunkt: Mit `push(noblock)` und einem 4-Word-FIFO (2 Paare) werden bei 5 s Poll-Intervall 373 von 375 Messungen pro Fenster verworfen. Es zaehlt nur das erste Paar — ein Ausreisser genuegt fuer einen Fehler.

Loesung: kontinuierlicher Ringpuffer mit Median. Ein modulinterner 25 ms-Timer leert den FIFO staendig — schneller als er vollaeuft (bei 75 Hz: alle 27 ms voll). Jede valide Messung (HIGH > 0 und LOW > 0) landet im Ringpuffer:

Listing 5: 25 ms Drain-Timer und Ringpuffer-Akkumulation

```
_BUF_SIZE = 30        # ~400 ms Datenfenster @ 75 Hz
_high_buf = [0] * _BUF_SIZE
_low_buf  = [0] * _BUF_SIZE
_buf_idx  = 0
_buf_count = 0
_reads_total     = 0   # Gesamtzahl gelesener Paare (Statistik)
_reads_discarded = 0   # davon verworfen (h=0, l=0 oder zu kurz)

def _drain_fifo(t=None):
    global _buf_idx, _buf_count, _last_update_us
    global _reads_total, _reads_discarded
    while _sm.rx_fifo() >= 2:
        x_high = _sm.get()
        x_low  = _sm.get()
        h = (0xFFFFFFFF - x_high) * 2
        l = (0xFFFFFFFF - x_low)  * 2
        _reads_total += 1
        if h > 0 and l > 0 and (h + l) > 100:  # Ausreisser verwerfen
            _high_buf[_buf_idx] = h
            _low_buf[_buf_idx]  = l
            _buf_idx = (_buf_idx + 1) % _BUF_SIZE
            if _buf_count < _BUF_SIZE:
                _buf_count += 1
            _last_update_us = utime.ticks_us()
        else:
            _reads_discarded += 1

# Interner Timer: 6 ms ~ 50% Zyklusperiode @ 75 Hz (adaptiv anpassbar)
_drain_timer = Timer()
_drain_timer.init(period=6, mode=Timer.PERIODIC,
                  callback=_drain_fifo)
```

Die Auswertung nutzt den **Median** der gesammelten Werte:

Listing 6: Median-Berechnung aus Ringpuffer

```
n = _buf_count
high_med = sorted(_high_buf[:n])[n // 2]
low_med  = sorted(_low_buf[:n])[n // 2]
T    = high_med + low_med
freq = round(1_000_000.0 / T, 2)    # Hz
duty = round(high_med / T * 100, 2) # Prozent
```

Warum Median statt Mittelwert? Der Median ist unempfindlich gegen Ausreisser: Ein einzelner HIGH = 0-Wert unter 30 Messungen veraendert den Median nicht. Der Mittelwert wuerde hingegen nach unten gezogen. Bei 300 gesammelten Messwerten pro 5 s-Fenster ist der Median statistisch sehr stabil — auch waehrend Anlauf und Lastwechseln.

7 Vergleich aller Methoden

Methode	Max. Freq.	Debounce	CPU	Build	Robust
Python IRQ	∼2 kHz	noetig	hoch	nein	gering
C-Natmod IRQ	∼10 kHz	noetig	mittel	.mpy	mittel
Core 1 Polling	∼250 kHz	adaptiv	Core 1	Firmware	mittel
PIO @ 1 MHz	250 kHz	nicht noetig	null	nein	hoch
PIO + Median	250 kHz	nicht noetig	null	nein	sehr hoch

8 Drop-in-Ersatz ohne Aenderung an main.py

Das Modul `pwmfeedback_pio.py` implementiert dieselbe API wie alle Vorgaenger:

Listing 7: Umschalten in main.py: eine Zeile

```
# vorher:
import pwmfeedback_core1 as pwmfeedback    # Core 1 Polling
# nachher:
import pwmfeedback_pio as pwmfeedback      # PIO + Median-Filter
```

`init_feedback_pin()` startet State Machine und Drain-Timer, `get_pump_feedback()` liefert dasselbe Dict mit `PIN5_Freq_Hz`, `PumpDuty` und `PumpStatus`. Der Rest von `main.py` — MQTT, Watchdog, Rampe, Boost — bleibt unveraendert.

Deploy per WebREPL, kein BOOTSEL noetig:

Listing 8: Deploy ohne Firmware-Flash

```
./deploy.sh pwmfeedback_pio.py
# -> WebREPL-Upload + MQTT-Reset, sofort aktiv
```

9 Logic Analyzer: EMI-Charakterisierung per PIO

Um die EMI-Stoerungen direkt zu messen, wurde ein zweites PIO-Programm als Logikanalysator gebaut (`la_pin5.py`). Statt Pulse zu messen, sampelt es den Pin Bit fuer Bit mit `in_(pins, 1)` und schiebt je 32 Bit per Autopush in den FIFO. Python liest die Woerter, expandiert die Bits und sucht Flanken.

Zwei Abtastraten wurden verglichen:

125 MHz (8 ns/Sample) — Glitch-Diagnose:

Das Capture-Fenster betraegt $1300 \times 32 \times 8\,\text{ns} \approx 333\,\mu\text{s}$ — ein winziger Ausschnitt einer 13 333 µs-Periode. Fuer 2 Sekunden vollstaendige Erfassung waere ein Puffer von $\frac{2 \times 125\,000\,000}{32} \approx 7{,}8\,MWoerter = 31\,MB$ noetig — der Pico hat gesamt 264 KB. 125 MHz ist also ausschliesslich fuer Kurzzeit-Diagnose geeignet.

Ergebnis des 125 MHz-Captures:

Kenngroesse	Wert
Fenster	202 µs @ 125 MHz / 8 ns
HIGH-Flanken ≥ 10 µs	0 (keine echten HIGH-Phasen im Ausschnitt)
HIGH-Glitches (7–8 µs)	6 Spikes
LOW-Glitches (≈10 µs)	1 Spike
Glitch-Rate	58 % aller Ereignisse

Die EMI-Spikes haben eine Dauer von **7–8 µs**. Das erklaert das Versagen bei 125 MHz (8 ns Abtastrate): Jeder Spike trifft mit hoher Wahrscheinlichkeit eine `jmp(pin)`-Instruktion und beendet die LOW-Messung vorzeitig. Bei 1 MHz (2 µs/Zyklus) sind diese Spikes dagegen *nicht* vollstaendig unsichtbar — 7 µs entsprechen ca. 3 Zaehlzyklen. Der entscheidende Unterschied ist der **Median-Filter**: Bei 30 Messungen pro Puffer genuegen wenige Ausreisser nicht, um den Median zu verschieben.

24 kHz (41,7 µs/Sample) — Signalstruktur:

Um die echten 75 Hz-Flanken sichtbar zu machen, wurde die Abtastrate auf 24 kHz reduziert — 1500 × 32 Samples = **2,0 Sekunden** Fenster bei nur 6 KB Puffergroeße.

Kenngroesse	Gemessen (LA 24 kHz)
Frequenz	75,35 Hz
Duty-Cycle	32,7 %
HIGH-Dauer	Ø 4338 µs (min 1208, max 4374)
LOW-Dauer	Ø 8933 µs (min 4083, max 8999)
Glitches (<1 ms)	0 — unterhalb der 41 µs-Aufloesung

Das echte Wilo-Signal ist ein sauberes 75 Hz-Rechteck. Die 7–8 µs-Spikes liegen weit unter der 41 µs-Aufloesung und sind bei dieser Abtastrate vollstaendig unsichtbar.

Fazit: Hoehere Abtastrate sieht mehr Stoerungen — nicht weil das Signal schlechter wird, sondern weil die Messung empfindlicher ist. 1 MHz ist der Sweet-Spot: Ausreichend schnell fuer prazise 75 Hz-Messung, ausreichend langsam um Motorspikes unter 2 µs auszublenden, und mit Median-Filter robust gegen die gelegentlichen 7–8 µs-Ausreisser.

10 Messwerte im Betrieb

Gemessene Werte der Wilo iPWM-Pumpe im Normalbetrieb (v2.60, Median-Filter):

Kenngroesse	Wert
Frequenz	75,0 Hz (stabil, jede Messung)
Duty-Cycle Anlauf	1,9 % (Standby-Modus)
Duty-Cycle Betrieb	34–66 % (lastabhaengig)
Aufloesung	2 µs
Messungen pro 5 s	∼300 (Ringpuffer)
CPU-Last	0 % (reiner Hardware-PIO)

Die Wilo-Pumpe garantiert ein stabiles 75 Hz-Feedback-Signal. Der Duty-Cycle codiert den Betriebsstatus nach Wilo-Protokoll: 5–75 % Normalbetrieb (proportional zur Foerderleistung), 80 % Abnormal Running, 85–90 % Abnormal Function, 95 % Permanent Failure, 100 % Interface Damaged / Power OFF.

11 FIFO-Synchronisation: blockierendes push() ist Pflicht

Der erste Entwurf verwendete `push(noblock)` — wenn der FIFO voll ist, wird der aktuelle Wert einfach verworfen, die State Machine laeuft weiter. Das klingt defensiv, ist aber tatsaechlich gefaehrlich:

Der FIFO fasst 4 Words (2 vollstaendige HIGH/LOW-Paare). Wenn Python wegen einer `client.publish()`-Operation mehrere Millisekunden nicht lesen kann und der FIFO ueberlaeuft, wird *einer* von zwei zusammengehoerigen Werten verworfen — ohne zu wissen welcher. Der naechste `sm.get()` liest dann HIGH als LOW und umgekehrt. Der Puffer ist dauerhaft desynchronisiert; alle folgenden Duty-Werte sind falsch.

Loesung: beide `push()` blockieren. Wenn der FIFO voll ist, stalliert die State Machine an dieser Stelle. Es wird kein Wort verworfen. HIGH und LOW bleiben unveraenderlich als Paar — die Synchronisation ist durch PIO-Hardware garantiert, nicht durch Software.

12 DiscardPct: was 16 % Verwurf bedeuten

Mit blockierendem `push()` tritt ein neues Phaenomen auf. Wenn Python fuer ≥ 10 ms blockiert ist (z. B. `client.publish()` sendet ein groesseres JSON-Paket), kann der Soft-Timer `_drain_fifo()` nicht feuern. Der FIFO fuellt sich in dieser Zeit mit 2 Paaren (4 Words), die State Machine stalliert am dritten `push()` fuer HIGH. Die gesamte LOW-Phase des dritten Zyklus laeuft durch, waehrend der PIO stillsteht.

Wenn der Drain-Timer spaeter endlich laeuft:

1. HIGH des dritten Zyklus kommt aus dem FIFO (korrekt gemessen)
2. LOW-Messung startet — aber Pin ist schon wieder HIGH
3. `jmp(pin, "low_done")` feuert sofort: $l = 0$
4. Pruefbedingung `l > 0` schlaegt fehl — Paar wird verworfen

Bei 75 Hz und einem Publish-Zyklus alle 5 s entfaellt pro ca. 375 Messungen eine Gruppe auf dieses Stallereignis. Gemessen im Dauerbetrieb (UPTIME > 600 s, Pumpe im Regelbetrieb):

Kenngroesse	**Wert**
Duty-Cycle	22,9 % (stabil)
Frequenz	75,0 Hz
Pufferfuellung N	30/30
DiscardPct	16,2 % (stabil)

Ursache: Die MicroPython-Laufzeit kann Soft-Timer nicht praeemptiv unterbrechen, wenn Core 0 durch C-Ebenen-I/O (TCP-Stack, MQTT-Publish) blockiert ist. Das ist ein architektonisches Merkmal des MicroPython-Single-Core-Betriebs — kein Fehler im PIO-Programm und kein Problem mit dem Wilo-Signal.

Messqualitaet ist nicht beeintraechtigt. Die verworfenen Paare ($l = 0$) werden vor dem Median-Puffer gefiltert und gehen nie in die Duty-Berechnung ein. Alle 30 Eintraege im Ringpuffer sind valide Messungen — die gemessenen 22,9 % Duty sind korrekt.

DiscardPct ist daher eine Systemkennzahl, die die Lastsituation von Core 0 sichtbar macht — kein Signal schlechter Messqualitaet. Sie wird als Telemetriewert in das MQTT-JSON exportiert:

Listing 9: Neue Telemetriefelder im MQTT-Payload

```
"PIN5_N":      30,      # Pufferfuellung (valide Eintraege im Ringpuffer)
```

```
"DrainMs":      6,      # aktuelles Drain-Intervall (ms)
"DiscardPct": 16.2,     # verworfene Flanken in Prozent
```

Ein dauerhaft hoher Wert ($> 25\,\%$) waere ein Hinweis auf ungewoehnlich lange MQTT-Blocker oder haengenden TCP-Stack. Unter normalen Betriebsbedingungen stabilisiert sich DiscardPct bei **ca. 15–17 %** — systembedingt und erwartet.

Code, Firmware & Deploy-Script: https://github.com/gerontec/wilo_pwm